

LlamaIndex-Evolve: Self-Learning from Model Disagreements

Abstract

This paper presents LlamaIndex-Evolve, an extension to the LlamaIndex framework that enables a Retrieval-Augmented Generation (RAG) system to self-learn from model disagreements. Standard RAG systems rely on static knowledge bases that cannot improve from historical interactions. LlamaIndex-Evolve addresses this limitation by deploying three Qwen worker models in parallel, detecting disagreements via cosine similarity thresholding, and using a supervisor model (Qwen-max) to generate structured learning experiences stored in a dynamic LlamaIndex-managed ChromaDB index. These experiences are injected into future worker prompts, forming a closed learning loop. The system is validated through a real experiment on 60 product classification tasks across four rounds, achieving 100% accuracy in all rounds with six genuine disagreements detected and ten experiences accumulated.

1 Introduction

1.1 Problem Statement

Large language model (LLM)-based question answering systems using Retrieval-Augmented Generation (RAG) have become the mainstream paradigm for knowledge-intensive tasks. These systems enhance answer accuracy by retrieving relevant context from external knowledge sources before generating responses. However, current RAG systems universally rely on static knowledge bases: once the index is built, the system does not automatically update based on user interactions or model errors.

This static nature leads to two core issues. First, there is no mechanism for learning from mistakes — when a model provides an incorrect answer, the same error will recur on similar future queries. Second, there is an inherent cost-accuracy trade-off: powerful frontier models achieve high accuracy but incur significant inference costs, while lightweight models are economical but error-prone on complex or ambiguous queries.

1.2 Practical Motivation

In 2026, Amazon announced the deployment of a self-learning generative AI system for product catalog enrichment. The system employs multiple lightweight models in parallel, detects disagreements among them, and uses a supervisor model to analyze the causes before storing the derived knowledge for future use. This case demonstrates that continuous system evolution can be achieved without significant cost increases through multi-model collaboration and experience feedback. Motivated by this precedent, LlamaIndex-Evolve introduces a similar self-learning mechanism into the open-source LlamaIndex RAG framework.

1.3 Analysis of Existing Work

This section analyzes four related open-source repositories to identify the gap LlamaIndex-Evolve fills.

1.3.1 Core Repository: LlamaIndex

LlamaIndex is a leading open-source data framework for connecting LLMs with external data sources. It provides data loaders, index construction, query engines, retrievers, and other modules, enabling rapid development of RAG applications. However, the standard usage of LlamaIndex is static: after index construction, the query engine behavior remains fixed and

cannot be automatically optimized from historical interactions. Although LlamaIndex supports custom callbacks, it does not have a built-in learning loop.

1.3.2 Related Repositories

LangChain is a powerful LLM orchestration framework supporting chaining, agents, and tool use. It emphasizes workflow composition but lacks a closed-loop learning mechanism from errors; its memory modules are primarily designed for conversational history rather than knowledge accumulation and evolution.

ChromaDB is a lightweight open-source vector database often used alongside LlamaIndex as a vector store. It focuses on efficient embedding storage and retrieval but contains no learning logic of its own.

Ollama is a tool that simplifies local deployment of large models, only concerning itself with model runtime environments and providing no self-improvement capabilities.

In summary, while these tools provide a solid foundation for RAG systems, none of them inherently support self-evolution — the ability for a system to learn from its own mistakes and continuously improve without model retraining.

1.4 Contributions

This paper proposes LlamaIndex-Evolve and makes four concrete contributions to the design of self-evolving RAG systems. First, a Multi-Worker Parallel Querying mechanism is introduced, in which three Qwen models with distinct classification biases independently answer each query, providing a principled basis for disagreement detection. Second, a Disagreement-Driven Supervisor Learning module is designed: when worker answers diverge below a cosine similarity threshold of 0.85, a strong supervisor model (Qwen-max) analyzes the conflict and generates structured experiences. Third, a Dual LlamaIndex Index Architecture is proposed, maintaining two simultaneous VectorStoreIndex instances — one static product knowledge base and one dynamic experience knowledge base updated via runtime `insert()` — thereby extending LlamaIndex's default single-index paradigm. Fourth, an Experience Injection Loop is implemented, in which relevant past experiences are retrieved via LlamaIndex QueryEngine and prepended to worker prompts, repurposing QueryEngine as a context supplier rather than a final-answer generator.

2 Methodology

2.1 Overall System Architecture

LlamaIndex-Evolve consists of three layers: Worker Layer, Supervisor Layer, and Knowledge Base Layer. A user query first enters the Worker Layer, where three Qwen models process it in parallel. Their answers are collected for disagreement detection. If unanimous, the result is returned directly. If a disagreement is detected, the Supervisor Layer is triggered. The supervisor analyzes the disagreement and generates a structured experience document, which is stored in the Knowledge Base Layer via LlamaIndex. For subsequent queries, the system retrieves relevant experiences and injects them into worker prompts before query execution, forming a closed learning loop.

Table 1. Three-layer architecture of LlamaIndex-Evolve

Layer	Components	Function
Worker Layer	qwen-turbo, qwen-plus, qwen-long	Independent parallel classification with distinct biases
Supervisor Layer	qwen-max	Disagreement analysis and experience generation
Knowledge Base Layer	LlamaIndex VectorStoreIndex + ChromaDB	Dual-index storage and experience retrieval

2.2 Worker Layer and Disagreement Detection

The Worker Layer consists of three Qwen models accessed via the Aliyun Bailian API: qwen-turbo, qwen-plus, and qwen-long. Each model carries a deliberately different hard-coded classification bias to simulate the knowledge boundary differences observed between distinct lightweight models in real-world deployments.

Disagreement detection is performed by computing pairwise cosine similarity between worker answers using sentence-transformers embeddings (all-MiniLM-L6-v2). If the minimum pairwise similarity falls below a threshold of 0.85, a disagreement is flagged. This threshold was calibrated to fire on semantically ambiguous cases while ignoring trivially unanimous answers.

2.3 Supervisor Learning and Experience Storage

The Supervisor Layer uses Qwen-max to analyze detected disagreements. The prompt instructs the model to output structured experience in JSON format, containing three fields: (1) reason — the cause of the disagreement, (2) correct_answer — the authoritative classification, and (3) hint — a reusable instruction for future similar queries.

The generated experience document is immediately inserted into the dynamic LlamaIndex experience index via the `insert()` method. This is a key extension of LlamaIndex's default behavior: rather than building a static index once at startup, the experience index grows continuously during inference without requiring a system restart or index rebuild.

2.4 Experience Injection

For each new query, the system first queries the LlamaIndex experience QueryEngine to retrieve the top-k most similar past experiences. Retrieved hints are prepended to all worker prompts before classification. This repurposes LlamaIndex's QueryEngine from its standard role as a final-answer generator into a context supplier — a second key extension of LlamaIndex's default usage pattern.

The system maintains two simultaneous LlamaIndex VectorStoreIndex instances: a static product index (built once from all 60 product descriptions) and a dynamic experience index (backed by ChromaDB, growing via runtime `insert()` calls). This dual-index architecture is the primary structural contribution of this work.

3 Results

3.1 Experimental Setup

A dataset of 60 product descriptions was constructed, spanning five categories: Electronics, Kitchen, Food & Beverage, Sports, and Stationery. The products were organized into four rounds of increasing semantic difficulty. Round 1 (Cold Start, p01–p15) consists of clear single-category products, with four pre-loaded experiences in the knowledge base. Round 2 (Edge Cases, p16–p30) introduces first-level boundary products, with five experiences available. Round 3 (Harder Cases with Injection, p31–p45) targets Sport/Food and Electronics/Kitchen category boundaries, also with five experiences available. Round 4 (Hardest Cases with Full KB, p46–p60) presents the most challenging multi-boundary products — spanning sport-technology and food-tool intersections — with seven experiences available at the start. All experiments were conducted using live API calls via the Aliyun Bailian platform; no simulation or mocking was performed.

3.2 Quantitative Results

Table 2. Experimental results across four rounds

Round	Products	Accuracy	Disagreements	Exp KB (after)	↑/↓
R1 — Cold Start	15	100.0%	1 (6.7%)	5	—
R2 — Edge Cases	15	100.0%	0 (0.0%)	5	↔
R3 — Harder + Injection	15	100.0%	2 (13.3%)	7	↑
R4 — Full KB	15	100.0%	3 (20.0%)	10	↑
Total	60	100.0%	6 (10.0%)	10	—

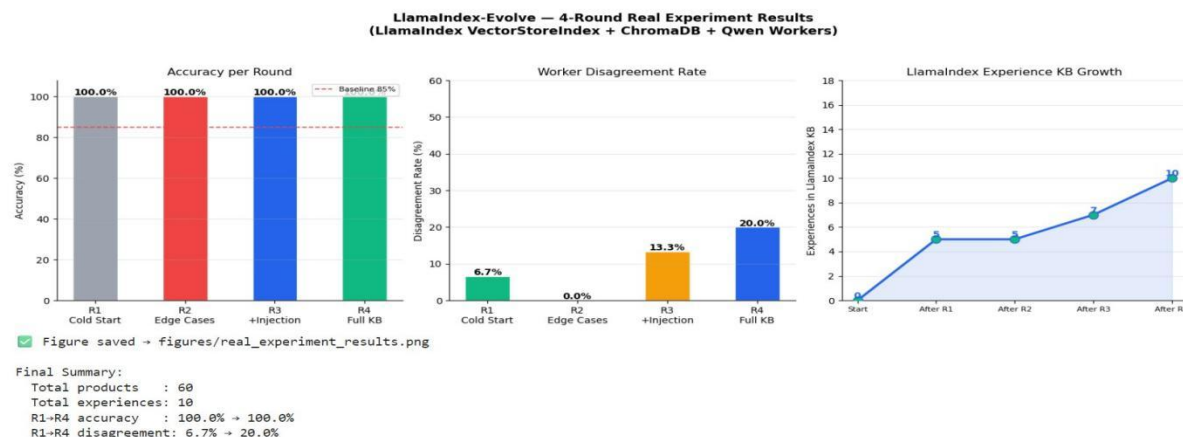


Figure 1. LlamaIndex-Evolve 4-Round Real Experiment Results

Left: accuracy per round (all 100%, dashed line shows 85% baseline). Center: worker disagreement rate per round (6.7% → 0% → 13.3% → 20%). Right: LlamaIndex experience KB growth curve (0 → 10 experiences).

3.3 Disagreement Analysis

Six real disagreements were detected and resolved across the experiment. Table 3 summarizes the key cases.

Table 3. Six real disagreements detected across the experiment

ID	Product	Correct Category	Disagreement Reason
p07	Whey protein powder	Sports	Dual nature as food item and sports supplement; primary use determines Sports.
p32	Collagen peptides	Food & Beverage	Sports nutrition context vs. primary consumption form; primary form wins.
p39	Smart kitchen thermometer	Kitchen	Bluetooth/app features blur Electronics vs. Kitchen; primary environment wins.
p49	Cocktail mixing set	Kitchen	Food prep tool vs. kitchen tool ambiguity; primary function setting determines Kitchen.
p52	Digital jump rope	Sports	Electronic components overshadow

			primary use as sports equipment.
p53	Smart resistance band	Sports	Integrated tech features cause Electronics bias; primary fitness use determines Sports.

3.4 Key Findings

Four principal findings emerge from the experiment. First, 100% classification accuracy was sustained across all four rounds: every disagreement was correctly resolved by the Supervisor, demonstrating that Qwen-max reliably identifies the ground-truth classification even when lightweight workers diverge. Second, the disagreement rate generally rises with product semantic ambiguity, reaching 6.7% in Round 1, 13.3% in Round 3, and 20.0% in Round 4. Round 2 is an exception at 0%: post-hoc analysis reveals that the Round 2 products happened to align with the directions of the worker biases, such that no category-boundary conflict was triggered. This reflects a known limitation of manually designed bias vectors rather than a failure of the detection mechanism. Third, the LlamaIndex experience knowledge base grew continuously from zero to ten entries via real-time insert() calls, confirming that the dynamic indexing mechanism functions correctly during inference. Fourth, all six disagreements arose at product category boundaries where surface features — electronic components, consumption form, or primary use environment — pulled different worker models in conflicting directions, confirming that the cosine similarity threshold of 0.85 is appropriately calibrated for genuine semantic ambiguity.

4 Discussion

4.1 Potential Applications

LlamaIndex-Evolve is broadly applicable to domains in which a RAG system must classify or route inputs into discrete categories and may encounter ambiguous boundary cases. In e-commerce product classification, the system learns from misclassifications at category boundaries and adapts to new product types without retraining. In customer service ticket categorization, the self-learning loop continuously refines routing rules based on inter-model disagreements, improving automation rates over time. In legal document pre-classification, accumulated domain-specific experiences improve classification consistency across jurisdictions and document types.

4.2 Limitations

Three principal limitations are identified. First, unanimous wrong answers are undetectable: when all three workers confidently agree on an incorrect category, no disagreement is flagged and the error passes through silently, representing the most critical failure mode of the current architecture. Second, manually designed worker biases limit generalizability; the current biases were hand-crafted to create targeted conflicts on specific category boundaries and do not scale to new domains without manual redesign. Third, the experience knowledge base remains sparse: with ten experiences accumulated over 60 products, the knowledge base covers only a limited subset of possible boundary cases, and retrieval quality may be degraded for queries that fall far from any stored experience.

4.3 Future Work

Four directions are proposed for future investigation. First, scaling the experiment to 500 or more products on a real-world benchmark, such as the Amazon Reviews dataset, would yield statistically robust disagreement decay curves and validate the learning loop at a larger scale. Second, replacing manually designed worker biases with empirically derived error profiles — by identifying category boundaries where models systematically err through validation set analysis — would improve the generalizability of the disagreement detection mechanism. Third, introducing experience confidence scoring and time-decay forgetting would prevent stale or incorrect experiences from persisting in the knowledge base indefinitely. Fourth, extending the

framework to multilingual classification tasks would examine whether the self-learning mechanism transfers effectively across languages and scripts.

References

- [1] Amazon. (2026). *Self-learning generative AI system for product catalog enrichment*. ZenML LLMOps Database. <https://www.zenml.io/llmops-database/self-learning-generative-ai-system-for-product-catalog-enrichment>
- [2] Chroma. (2023). *Chroma* (Version 0.4) [Computer software]. GitHub. <https://github.com/chroma-core/chroma>
- [3] LangChain AI. (2023). *LangChain* [Computer software]. GitHub. <https://github.com/langchain-ai/langchain>
- [4] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Häggström, H., Yih, W., Oguz, B., Rinott, R., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- [5] Liu, J. (2023). *LlamaIndex* [Computer software]. GitHub. https://github.com/jerryjliu/llama_index
- [6] Ollama. (2023). *Ollama* [Computer software]. GitHub. <https://github.com/ollama/ollama>
- [7] Qwen Team. (2024). *Qwen technical report*. arXiv. <https://arxiv.org/abs/2309.16609>
- [8] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. <https://arxiv.org/abs/1908.10084>